# Healthcare Insurance Eligibility Verification App: Complete Technical Brief for Staffingly, Inc.

**Staffingly's 800+ healthcare provider clients need a standalone eligibility verification tool that matches or exceeds what Jindal Healthcare's HealthX platform delivers — RPA-driven automation handling 70%+ of verifications,** (Jindal Healthcare) **AI-powered benefits interpretation, and real-time payer connectivity — while being purpose-built for virtual staffing workflows.** This brief provides every technical specification, API reference, architecture decision, and sprint-by-sprint build plan a development team needs to start building immediately. The market is moving from simple EDI 270/271 lookups toward AI-powered, multi-stage verification across the entire revenue cycle, and this app must be positioned at the forefront of that shift.

---

## A) Concept and features: what the app must do

The app ("StaffVerify") functions as an all-in-one eligibility verification command center serving Staffingly's healthcare provider clients. It combines automated real-time eligibility checks, AI-powered benefits interpretation, RPA-based payer portal navigation, and human-in-the-loop workflows — all accessible through a multi-tenant SaaS dashboard.

**Core feature set, informed by competitive analysis of Availity, Waystar, Experian Health, Change Healthcare, Trizetto, Inovalon, and Jindal Healthcare's HealthX platform:**

**Real-time eligibility verification** sends X12 EDI 270 transactions through clearinghouses and returns parsed 271 responses in under 5 seconds. The system supports both single-patient real-time checks and batch verification for next-day appointment schedules (overnight runs for all scheduled patients). Coverage status, plan details, copays, deductibles, coinsurance, out-of-pocket maximums, and prior authorization requirements (Pilotfishtechnology) display in a normalized, consistent format regardless of payer. (Hipaaatlas) (1edisource) The system routes through EDI first, falls back to FHIR CoverageEligibilityRequest where supported, and uses RPA portal scraping only when electronic channels are unavailable.

**Insurance card OCR and data capture** uses Azure AI Document Intelligence's pre-built health insurance card model (Microsoft Learn) (v4.0 GA) to extract insurer name, member ID, group number, plan type, RxBIN, RxPCN, and dependent information from phone-captured images or scans. (Microsoft Learn) This eliminates manual data entry at registration and feeds directly into the verification workflow.

**Coverage discovery** searches across commercial and government payers to find previously unknown primary, secondary, or tertiary coverage for self-pay patients — a feature that Waystar, Experian Health, and Inovalon all offer as premium capabilities. (Waystar) This alone can recover **$18M+ in potential denials** per large health system annually (Experian Health case study with Providence Health).

**AI-powered benefits interpretation** uses Claude or GPT-4 via RAG pipelines to translate raw 271 response data into plain-language benefits summaries, calculate patient responsibility estimates for specific CPT codes,

and flag coverage anomalies. The system maintains a vector database of payer-specific rules, plan documents, and clinical policies for retrieval-augmented generation.

**RPA payer portal automation** deploys unattended bots (UiPath or Automation Anywhere) to navigate payer web portals for verifications that cannot be completed via EDI or FHIR. Bots authenticate, input patient demographics, submit inquiries, scrape structured data from response pages, and write results back to the system. Superblocks auxiliobits A modular architecture separates portal-specific navigation from business logic for maintainability.

**Human-in-the-loop (HITL) workflow** routes low-confidence verifications (below 85% AI confidence) to Staffingly's virtual staff for manual review. The queue prioritizes by appointment time, dollar amount, and SLA urgency. Every human decision feeds back into the AI training pipeline. Beetroot

**Multi-tenant client management** supports 800+ provider organizations with isolated data, custom workflows per client SOP, payer-specific rules, and white-label branding options. Each client can configure their own verification triggers, escalation rules, and reporting preferences.

**EHR/EMR integration** supports bidirectional data exchange with Epic, Athenahealth, eClinicalWorks, Cerner/Oracle Health, NextGen, DrChrono, Kareo/Tebra, AdvancedMD, and Practice Fusion via FHIR R4 APIs, HL7 v2 ADT message listeners, and direct REST APIs. SMART on FHIR embedded apps allow clinicians to verify eligibility without leaving their EHR workflow. Microsoft Learn

**Dashboard and reporting** provides real-time operational dashboards (verification queues, payer response times, exception tracking) and executive analytics (verification rates, denial trends, cost savings, staff productivity) via embedded React dashboards and Power BI integrations.

**Additional features** include automated pre-authorization checks, Medicare MBI lookup/validation, coordination of benefits (COB) detection, auto-recheck when patient data changes, configurable alerts for staff, and detailed audit logging for HIPAA compliance.

---

## B) Technical architecture: the complete stack

### Recommended technology stack

| Layer | Technology | Rationale |
|---|---|---|
| **Frontend** | React 18+ with TypeScript, Material-UI | Largest healthcare ecosystem, component libraries, SMART on FHIR client support |
| **Backend API** | Node.js/NestJS (real-time API layer) | High throughput for I/O-bound EDI processing, excellent async handling |

| Layer | Technology | Rationale |
|---|---|---|
| **Batch Processing** | Python/FastAPI | Rich data processing libraries, ML integration, EDI parsing (TigerShark, PyX12) |
| **Database (Primary)** | PostgreSQL on AWS Aurora | HIPAA-eligible, encrypted at rest/transit, relational for patient/eligibility data |
| **Database (Document)** | MongoDB on AWS DocumentDB | Flexible schema for varied 271 response structures across payers |
| **Cache** | Redis on AWS ElastiCache | Caching frequently verified patients, session management, HIPAA-eligible |
| **Message Queue** | Amazon SQS + SNS | HIPAA-eligible, dead-letter queues for failed verifications, pub/sub for events |
| **Event Streaming** | Apache Kafka on Amazon MSK | High-throughput audit log streaming, event-driven architecture |
| **Search** | Elasticsearch (OpenSearch) | Full-text search across eligibility records, audit logs |
| **Cloud** | AWS (primary) | Most mature HIPAA offering, 166+ eligible services, Amazon Web Services HealthLake for FHIR |
| **CDN/WAF** | CloudFront + AWS WAF | DDoS protection, OWASP Top 10, geographic filtering |
| **API Gateway** | AWS API Gateway or Kong | Rate limiting, authentication, request routing, logging |
| **Container Orchestration** | AWS EKS (Kubernetes) | HIPAA-eligible, auto-scaling, pod security standards |
| **CI/CD** | GitHub Actions + ArgoCD | GitOps-based deployment, infrastructure as code |
| **IaC** | Terraform | Multi-cloud support, version-controlled infrastructure |
| **Monitoring** | OpenTelemetry + Grafana + CloudWatch | Distributed tracing, metrics, structured logging |
| **Vector Database** | Pinecone or FAISS | RAG pipeline for payer-specific rules retrieval |

### Microservices architecture

The system decomposes into **eight core microservices**, each independently deployable:

1. **Patient Intake Service** — receives patient demographics, insurance card data (via OCR), appointment triggers from EHR webhooks and ADT messages
2. **Eligibility Engine** — routes verification requests through the optimal channel (EDI 270/271 → FHIR → RPA), manages retries and failover
3. **EDI Gateway** — generates X12 270 transactions, submits to clearinghouses (Availity, Change Healthcare), parses 271 responses using `node-x12` or Stedi
4. **FHIR Adapter** — handles CoverageEligibilityRequest/Response via FHIR R4, SMART on FHIR authorization flows
5. **RPA Orchestrator** — manages UiPath/Automation Anywhere bot fleet for payer portal navigation, handles credential management and bot health monitoring
6. **AI Interpreter** — runs Claude/GPT-4 inference for benefits interpretation, confidence scoring, and patient responsibility calculation via RAG pipelines
7. **HITL Queue Manager** — manages human review queues, priority scoring, task assignment, and feedback collection for model retraining
8. **Notification & Reporting Service** — sends alerts, generates dashboards, streams events to Power BI, manages client-specific reporting

**End-to-end data flow**

```
[EHR/PM System] → ADT^A04 or Webhook (patient registered/appointment created)
    ↓
[Patient Intake Service] → extracts PID + IN1 segments or FHIR Patient/Coverage
    ↓
[Eligibility Engine] → determines optimal verification route per payer
    ├─ Route 1: EDI Gateway → X12 270 → Clearinghouse → Payer → 271 (1-5 sec)
    ├─ Route 2: FHIR Adapter → CoverageEligibilityRequest → Payer FHIR API
    └─ Route 3: RPA Orchestrator → Bot → Payer Portal → Screen Scrape
    ↓
[Response Parser] → normalizes all responses to unified JSON schema
    ↓
[AI Interpreter] → benefits interpretation, confidence scoring, patient cost estimate
    ├─ High confidence (>90%): auto-approve → write to DB → notify frontend
    ├─ Medium (70-90%): flag for quick review → HITL Queue
    └─ Low (<70%): full human review → HITL Queue
    ↓
[Database] → PostgreSQL (structured) + DocumentDB (raw responses) + Redis (cache)
    ↓
[API Layer] → REST endpoints → React Dashboard + SMART on FHIR EHR widget
```

### Caching strategy

Cache keys use a hash of (member_id + payer_id + service_date + service_type_code). Basic coverage status caches for **24 hours**; benefit details cache for **4-6 hours**. The system pre-fetches eligibility for all next-day appointments via overnight batch runs and implements stale-while-revalidate — serving cached results immediately while triggering async refreshes. Cache invalidation fires on enrollment change feeds or patient demographic updates.

### Error handling architecture

The system implements a **circuit breaker pattern** — after 5 consecutive failures to a specific payer endpoint, the circuit opens for 10 minutes before half-opening to test recovery. Failed transactions after 3 retries (exponential backoff: 2s, 4s, 8s) route to an SQS dead-letter queue for manual reprocessing. Every EDI transaction uses unique TRN trace numbers for idempotency and response correlation. The system returns 999 Implementation Acknowledgments for syntax errors (MVP Health Care) and maps AAA error segments from 271 responses to human-readable error codes.

---

## C) AI and ML components: models, training data, and pipelines

### Insurance card OCR pipeline

**Azure AI Document Intelligence** (v4.0 GA) provides the pre-built health insurance card model via (POST /documentIntelligence/documentModels/prebuilt-healthInsuranceCard.us:analyze). This extracts insurer name, member ID, member name, group number, RxBIN, RxPCN, plan type, and dependent information from phone-captured card images (Microsoft Learn) with no custom training required. For practices not on Azure, **AWS Textract** with AnalyzeDocument queries ("What is the member ID?") serves as the alternative, feeding into Amazon Comprehend for entity classification. (AWS) **GPT-4o's multimodal capabilities** offer a third path — processing card images directly with **90.5% field accuracy** but slower throughput (~33 seconds/page), (Businesswaretech) making it best for fallback scenarios.

### Benefits interpretation via LLM + RAG

The AI interpretation layer uses **Claude 3.5 Sonnet or GPT-4** with retrieval-augmented generation to transform raw eligibility data into actionable insights. The RAG pipeline ingests payer manuals, benefit summaries, provider contracts, fee schedules, and clinical policies into chunks, embeds them using **BGE-M3** (which achieved 97.6% (PubMed Central) retrieval accuracy in healthcare RAG benchmarks), and stores vectors in Pinecone or FAISS. On each eligibility query, the system retrieves relevant payer-specific rules and generates grounded interpretations.

Research shows GPT-4 with RAG achieves **96.4% accuracy** in medical guideline interpretation versus 86.6% human accuracy (Nature) (Nature, 2025), and RAG reduces hallucination rates by over 40% versus standalone LLMs. (Frontiers)

**System prompt for the eligibility interpretation agent:**

> You are a certified healthcare eligibility specialist. Given a parsed EDI X12 271 response and relevant payer plan documents, extract and interpret:
>
> 1. Coverage status (Active/Inactive/Terminated) with effective dates
> 2. Plan type (HMO/PPO/EPO/POS/HDHP) and network
> 3. In-network deductible (individual/family) — amount and remaining
> 4. Out-of-pocket maximum — amount and remaining
> 5. Copays by service type: PCP, specialist, ER, urgent care, imaging, lab
> 6. Coinsurance percentages (in-network and out-of-network)
> 7. Prior authorization requirements for the scheduled service
> 8. Coordination of benefits / secondary coverage indicators
> 9. Patient estimated responsibility for the specific CPT code(s)
>
> Output structured JSON. Set any undetermined field to null with explanation.
> Flag any coverage anomalies, potential denials, or data inconsistencies.
> Never fabricate coverage details — state "unable to determine" when uncertain.

### ML models for predictive analytics

Three supervised learning models operate in the pipeline. A **denial prediction model** (XGBoost or LightGBM) trained on historical 271 data, claims history, and payer behavior predicts likelihood of coverage-related denials before claim submission. A **propensity-to-pay model** (similar to Jindal's HealthX) (Jindal Healthcare) prioritizes AR worklists by payment likelihood using patient demographics, payer mix, historical payment patterns, and coverage details. A **routing classifier** (logistic regression or random forest) determines whether each verification should go through EDI, FHIR, or RPA based on payer, coverage type, and historical success rates per channel.

### Confidence scoring and HITL routing

Every AI decision receives a confidence score using calibrated probability outputs. The tiered routing system sends **high-confidence results (>90%)** through straight-through processing, **medium-confidence (70-90%)** to a quick-review queue with AI-suggested actions, and **low-confidence (<70%)** to full human review. The 85% threshold is the recommended starting point, adjusted after 2-3 calibration rounds. (Brics-econ) Critically, AI confidence scores are **not shown to human reviewers** to prevent automation bias and rubber-stamping. (MindStudio)

---

## D) RPA and automation layer: tools, workflows, and bot instructions

### RPA platform selection

| Platform | Best For | Pricing | Healthcare Features |
|---|---|---|---|
| **UiPath** | Broadest healthcare marketplace | Enterprise pricing (expensive) | Myndshft eligibility (700+ payers, 93% US covered lives), UiPath Marketplace CMS-1500/UB-04 Document AI, NPI Registry, EDI Conversion Suite |
| **Automation Anywhere** | Cloud-native, cost-effective | ~$300/month per bot; free Community Edition | Gartner Magic Quadrant Leader 7 years running, 40% lower TCO than Blue Prism, Agentic Process Automation |
| **Microsoft Power Automate** | Microsoft-ecosystem shops | $15/user/month Premium | FHIRlink connector for Azure Health Data Services and Epic on FHIR, AI Builder for document processing |

**Recommendation:** Start with **Automation Anywhere** for cost efficiency and cloud-native architecture, with UiPath as the scale-up option for its deeper healthcare marketplace. Use Power Automate for lightweight integrations within Microsoft-heavy client environments. SS&C Blue Prism

### Bot architecture

Deploy a **hybrid attended/unattended model**. Unattended bots run 24/7 on virtual machines executing overnight batch verifications for next-day appointments, downloading remittance files, DigitalStaff and running coverage discovery sweeps. Attended bots assist front-office staff during business hours with real-time verification exceptions triggered by patient check-in events.

### Payer portal bot workflow

1. Bot receives verification request from Eligibility Engine (patient demographics + insurance info)
2. Bot selects appropriate payer portal from portal registry:
   - UnitedHealthcare → uhcprovider.com
   - Aetna → availity.com (Aetna channel)
   - Cigna → cignaforhcp.cigna.com
   - Humana → availity.com (Humana channel)
   - BCBS → state-specific portals or Availity
   - Medicare → CMS HETS
   - Medicaid → state-specific portals
3. Bot authenticates with stored credentials (managed via HashiCorp Vault)
4. Bot navigates to eligibility lookup form using CSS/XPath selectors
5. Bot inputs: patient name, DOB, member ID, SSN last 4 (if required), service date

6. Bot submits inquiry, waits for response page (timeout: 30 seconds)
7. Bot extracts structured data: coverage status, copays, deductibles, coinsurance, out-of-pocket max, prior auth requirements, PCP, network status, COB info
8. Bot writes structured JSON to Eligibility Engine
9. Bot logs complete audit trail (screenshots on failure)
10. On failure: retry once → if still failed, route to HITL queue with error context

### Building resilient bots

Portal redesigns are the primary failure mode for RPA. (Nanonets Health) Mitigate this through a **modular page-object architecture** that separates portal-specific navigation selectors from business logic — only navigation modules need updating when portals change. Use a **multi-selector fallback hierarchy** (element ID → CSS selector → XPath → image matching) for each interaction point. Implement **self-healing automation** using AI-powered selector repair (available in UiPath and Automation Anywhere). Set up automated monitoring that detects bot failures within 5 minutes and triggers alerts. Maintain **version-controlled selector libraries** per payer portal with automated regression testing.

### MFA and CAPTCHA handling

Many payer portals now require MFA. Dedicated phone numbers for SMS codes or authenticator app integration (TOTP) handle standard MFA flows. For CAPTCHAs, the recommended approach is to **avoid portals with aggressive CAPTCHA** and use EDI/API channels instead. Where portals are the only option, some RPA tools integrate with CAPTCHA-solving services, though this raises legal and ethical questions. The long-term solution is the industry's migration to FHIR APIs, which eliminates portal scraping entirely.

---

## E) Payer integration guide: EDI, FHIR, and portal access

### X12 EDI 270/271 transaction structure

The **ASC X12N 005010X279A1** standard governs eligibility transactions. (IntuitionLabs +2) A 270 (Eligibility Inquiry) contains hierarchical loops: **2000A** (Information Source/Payer with NM1*PR segment), **2000B** (Information Receiver/Provider with NM1*1P and NPI), **2000C** (Subscriber with NM1*IL containing member ID, DMG with DOB, DTP with service date, and EQ with service type code 30 for general health benefit plan coverage), and optional **2000D** (Dependent). (IntuitionLabs) (IntuitionLabs)

The 271 (Eligibility Response) mirrors this hierarchy and adds critical **EB (Eligibility/Benefit) segments** containing coverage status (active/inactive), service type codes, coverage level, insurance type, monetary amounts (copay, coinsurance, deductible), and time period qualifiers. (IntuitionLabs) **AAA segments** carry error codes when inquiries fail (Pilotfishtechnology) (member not found, invalid data). (Intuitionlabs) **DTP segments** contain coverage effective/termination dates. (Intuitionlabs) **MSG segments** carry free-form payer messages.

### Clearinghouse connectivity

| Clearinghouse | Connections | Protocol | Best For |
|---|---|---|---|
| **Availity** | 2,000+ payers, 3M+ providers | SFTP/AS2/HTTPS, CORE-certified | Widest payer reach; free basic tier |
| **Change Healthcare (Optum)** | 2,400+ payers | REST API (JSON ↔ X12 translation) | Developer-friendly JSON API; rules engine |
| **Trizetto (Cognizant)** | 8,000+ payers | Gateway EDI | Broadest payer connections; 98% acceptance |
| **Stedi** | Cloud EDI platform | REST API | Modern API-first EDI; JSON conversion; free EDI Inspector |

**Recommended approach:** Primary clearinghouse is **Availity** for broadest payer reach and free basic tier. Secondary clearinghouse is **Change Healthcare** for its developer-friendly JSON REST API that translates X12 EDI internally. Use **Stedi** as the EDI parsing and validation layer — it provides end-to-end X12-to-JSON pipeline with EDI Inspector for debugging.

CAQH CORE rules mandate **90% of real-time 270 inquiries must receive a 271 response within 20 seconds**. (invene) (Intuitionlabs) In practice, typical clearinghouse API calls complete in **1-5 seconds**. (Invene)

### EDI parsing libraries by language

For **Node.js**: (node-x12) (parser, generator, query engine with streaming), (x12-parser) (transform stream-based, handles large files). (Michaelachrisco) For **Python**: (TigerShark) (X12 parser with 270/271 facade), (PyX12) (validation and translation). For **.NET**: (EdiFabric) (commercial HIPAA 5010 translator). (EdiFabric Docs) For a **managed service**: Stedi provides cloud-based parsing, JSON conversion, and validation without maintaining libraries.

### FHIR API integration

CMS mandates FHIR R4-based Patient Access APIs for Medicare Advantage, Medicaid, CHIP, and QHP payers. (Microsoft Learn) The **CoverageEligibilityRequest** resource submits eligibility inquiries with patient reference, insurer reference, coverage reference, service date, and item details (category and CPT codes). (Hapifhir) The **CoverageEligibilityResponse** returns coverage status ((inforce: true/false)), benefit details (copay, coinsurance, deductible amounts), and authorization requirements. (HL7 International) SMART on FHIR authorization uses OAuth 2.0 with scopes like (patient/Coverage.read) and (patient/CoverageEligibilityRequest.write).

**Current adoption reality:** As of 2023-2024, **94%+ of eligibility verifications still flow through EDI 270/271**. FHIR adoption is growing rapidly, with CMS-0057-F requiring Prior Authorization FHIR APIs by

January 2027. Binariks invene The app must support both EDI and FHIR simultaneously via a unified eligibility engine.

### Major payer portal registry

| Payer | Portal | EDI Available | FHIR Available | RPA Needed |
|---|---|---|---|---|
| UnitedHealthcare | uhcprovider.com | ✅ | Partial | Fallback |
| Aetna | availity.com (Aetna channel) | ✅ | Partial | Fallback |
| BCBS (varies by state) | State-specific or Availity | ✅ | Varies | Some states |
| Cigna | cignaforhcp.cigna.com | ✅ | Partial | Fallback |
| Humana | availity.com (Humana channel) | ✅ | Partial | Fallback |
| Medicare | CMS HETS | ✅ | ✅ (Blue Button 2.0) | Rarely |
| Medicaid | State-specific | ✅ (varies) | Varies | Often needed |

### Legal considerations for portal automation

Most payer portals explicitly prohibit automated/bot access in their Terms of Service. While enforcement against healthcare RPA is rare, the legal risk exists under the Computer Fraud and Abuse Act. **Best practice: use EDI/API channels for all payers that support them; reserve RPA exclusively for payers lacking electronic alternatives.** The industry push toward FHIR APIs is partly to eliminate portal scraping dependency.

---

## F) HIPAA and security requirements: encryption, auditing, and access controls

### Encryption requirements

All ePHI must be encrypted with **AES-256 at rest** (AWS KMS-managed keys) and **TLS 1.2+ in transit** (TLS 1.3 preferred). Encrypted ePHI that is breached qualifies for HIPAA's **safe harbor** — it is not reportable under the Breach Notification Rule. HIPAA Guide Implement **field-level encryption** using AES-256-GCM for the most sensitive fields: SSN, member ID, and DOB. Database-level encryption (Aurora's storage encryption + TDE) provides the baseline, with field-level encryption layered on top for defense-in-depth.

Note: HHS proposed in December 2024 to eliminate the distinction between "required" and "addressable" HIPAA specifications, HHS.gov making **all encryption mandatory** (no longer addressable). Veeam Build to the mandatory standard now.

### Access control implementation

Every user requires a **unique identifier** (no shared accounts). HHS.gov Moesif Implement role-based access control (RBAC) with least-privilege principles. Roles include: Provider Admin (full access to own organization's data), Verification Staff (read/write eligibility records), Reporting User (read-only dashboards), System Admin (infrastructure management, no PHI access). Enforce **MFA for all ePHI access** — the proposed 2024 NPRM makes this explicitly mandatory. Implement automatic session logoff after **15 minutes** of inactivity. HHS.gov Emergency access procedures must be documented for system failures.

### Audit logging

Log every interaction with ePHI: user identity, timestamp, action (create/read/update/delete), resource accessed, source/destination of transfers, and success/failure status. Censinet Atlantic.Net Use **tamper-proof storage** (WORM or cryptographic hashing) Censinet with a minimum **6-year retention period** aligned with HIPAA §164.316(b)(2)(i). I.S. Partners Stream audit logs to Amazon MSK (Kafka) for real-time processing and archive to S3 Glacier after 1 year. Conduct quarterly audit log reviews.

### Business Associate Agreements

Execute BAAs with every vendor that touches ePHI: cloud provider (AWS via Artifact), Amazon Web Services clearinghouses (Availity, Change Healthcare), RPA platform provider, AI model providers (Anthropic, OpenAI — both offer BAAs), OCR provider (Azure), and all subcontractors. The proposed 2025 HIPAA updates require **annual written verification** of business associate safeguards. HHS.gov Online and On Point

### EDI security

All EDI transmissions must use **AS2** (S/MIME encryption + digital signatures + MDN receipts) for real-time or **SFTP** (SSH encryption with key-based authentication) for batch. IntuitionLabs Pi Register as a Trading Partner with each clearinghouse, complete connectivity testing, Blue Shield of California and maintain trading partner agreements defining security provisions.

### Network architecture

Deploy all PHI workloads in **private subnets** within a VPC. AWS No public IP addresses on any PHI-processing resources. Use VPC PrivateLink for database and service access. Deploy AWS WAF for OWASP Top 10 protection and AWS Shield for DDoS protection. Implement security groups with whitelist-only rules (deny all by default). Separate environments — **dev/staging must never contain real PHI**. Dash Solutions

### Compliance certification roadmap

**Months 1-3:** Execute cloud BAA, conduct HIPAA risk assessment, deploy encrypted VPC architecture, implement IAM with MFA/RBAC, enable audit logging, draft vendor BAAs. **Months 3-6:** Implement OAuth 2.0/OIDC, deploy WAF and API gateway, set up EDI connections via AS2/SFTP, complete trading partner agreements, begin automated vulnerability scanning. **Months 6-12:** Complete SOC 2 Type I audit (Security + Confidentiality, estimated $20K-$60K), Thoropass begin SOC 2 Type II observation period, consider HITRUST e1 assessment. **Year 2:** SOC 2 Type II certification ($30K-$100K), Drata HITRUST i1 or r2 certification.

## G) Developer build instructions: sprint-by-sprint plan

### Sprint 0: Foundation (Weeks 1-2)

Set up AWS Organization with HIPAA-compliant account structure. Execute BAA via AWS Artifact. (AWS) Deploy VPC with public/private subnet architecture using Terraform. Configure AWS KMS for encryption key management. Set up EKS cluster with pod security standards, RBAC, and network policies. Configure CI/CD pipeline (GitHub Actions + ArgoCD). Establish development environments with synthetic (non-PHI) test data. Set up OpenTelemetry + Grafana for observability.

**Deliverable:** HIPAA-compliant cloud infrastructure, CI/CD pipeline, development environments.

### Sprint 1: Data model and patient intake (Weeks 3-4)

Design and implement PostgreSQL schema: (patients) (demographics, MRN), (insurance_policies) (payer, member_id, group, plan_type, effective_dates), (eligibility_checks) (request_id, channel, status, response_data, confidence_score, created_at), (eligibility_responses) (parsed benefits: copay, deductible, coinsurance, OOP_max, auth_requirements), (audit_logs) (user, action, resource, timestamp). Implement field-level encryption for SSN and member_id columns. Build Patient Intake Service with REST endpoints: (POST /patients), (POST /patients/{id}/insurance), (GET /patients/{id}/eligibility). Implement insurance card OCR using Azure Document Intelligence health insurance card model.

**Deliverable:** Database schema, Patient Intake Service, insurance card OCR endpoint.

### Sprint 2: EDI 270/271 gateway (Weeks 5-7)

Build EDI Gateway microservice. Implement X12 270 transaction generation using (node-x12) library — constructing ISA/GS/ST/BHT envelopes, HL loops (payer→provider→subscriber), NM1 segments for payer/provider/subscriber, DMG for demographics, DTP for service dates, EQ for service type codes. (IntuitionLabs) Integrate with Availity as primary clearinghouse (register as trading partner, configure AS2/HTTPS connectivity, complete SNIP Level 1-2 compliance testing). Implement 271 response parsing — extract EB segments (coverage status, copays, deductibles, coinsurance), AAA error codes, DTP coverage dates, MSG free-form messages. (Proedi) (IntuitionLabs) Normalize all 271 responses to unified JSON schema. Implement 999/997 acknowledgment handling. Build retry logic with exponential backoff and circuit breaker pattern. Store raw and parsed responses in DocumentDB.

**Deliverable:** Working EDI 270/271 pipeline through Availity, response normalization engine.

### Sprint 3: FHIR adapter and multi-channel routing (Weeks 8-9)

Build FHIR Adapter microservice using (fhirclient) JavaScript library. Implement CoverageEligibilityRequest/Response handling per FHIR R4 spec. Configure SMART on FHIR OAuth 2.0 flows (EHR launch, standalone launch, backend services). Build the Eligibility Engine routing logic: check

payer capability registry → route to EDI (default) → fallback to FHIR → fallback to RPA. Implement channel preference configuration per payer. Add response aggregation for multi-payer patients (primary/secondary/tertiary parallel queries).

**Deliverable:** FHIR eligibility flow, multi-channel routing engine, multi-payer support.

**Sprint 4: Frontend dashboard v1 (Weeks 10-12)**

Build React frontend with TypeScript and Material-UI. Implement core views: patient search, eligibility check initiation, verification results display (coverage status, plan details, patient responsibility breakdown), batch verification queue, exception/HITL review queue. Build real-time updates via WebSocket connections. Implement RBAC in the UI (role-based view restrictions). Build multi-tenant infrastructure (organization selector, data isolation). Implement the insurance card upload flow (camera capture → OCR → auto-fill → verify).

**Deliverable:** Functional web dashboard with real-time eligibility verification.

**Sprint 5: RPA automation layer (Weeks 13-15)**

Deploy Automation Anywhere cloud environment. Build payer portal bot templates for top 5 payers (UnitedHealthcare, Aetna, BCBS, Cigna, Humana). Implement credential management via HashiCorp Vault. Build modular page-object architecture with multi-selector fallback hierarchy. Implement attended and unattended bot deployment. Build batch scheduling for overnight verification runs. Implement screenshot-on-failure debugging. Build bot health monitoring and alerting. Integrate RPA Orchestrator microservice with Eligibility Engine.

**Deliverable:** RPA bots for top 5 payers, batch automation, bot monitoring.

**Sprint 6: AI interpretation engine (Weeks 16-18)**

Build AI Interpreter microservice. Integrate Claude API (via Anthropic) or GPT-4 API (via OpenAI) with BAA. Build RAG pipeline: ingest payer plan documents and clinical policies → chunk → embed with BGE-M3 → store in Pinecone. Implement prompt templates for 271 interpretation, benefits summarization, patient cost estimation, and anomaly detection. Build confidence scoring system with calibrated probability outputs. Implement tiered HITL routing (high/medium/low confidence). Build the human review interface with approve/edit/reject actions and structured feedback capture. Implement feedback-to-training pipeline.

**Deliverable:** AI-powered benefits interpretation, confidence scoring, HITL workflow.

**Sprint 7: EHR integrations (Weeks 19-22)**

Build SMART on FHIR embedded app for Epic and Cerner (registered on fhir.epic.com and code Console). Implement Athenahealth direct API integration (`POST/GET /v1/{practiceid}/insurances/{insuranceid}/eligibility`). Build HL7 v2 ADT listener using Mirth Connect for ADT^A04 (Register Patient) and ADT^A08 (Update Patient) triggers — extract PID + IN1 segments → construct 270. Build webhook receivers for DrChrono

(appointments created/modified) and eClinicalWorks event triggers. Test against each EHR's sandbox environment.

**Deliverable:** Live EHR integrations with Epic, Athenahealth, Cerner, eCW, DrChrono.

**Sprint 8: Dashboards, reporting, and coverage discovery (Weeks 23-25)**

Build executive dashboard (React + Recharts): verification rate, denial rate trends, cost savings, payer performance comparison, month-over-month trends. Build operational dashboard: real-time verification queue, payer response heatmap, exception tracking, staff productivity metrics. Integrate Power BI for advanced analytics (connect to PostgreSQL via DirectQuery). Implement coverage discovery engine — systematic cross-payer search for self-pay patients using batch 270 inquiries across major commercial and government payers. Build automated pre-authorization requirement detection.

**Deliverable:** Full reporting suite, coverage discovery, pre-auth detection.

**Sprint 9: Hardening, compliance, and launch prep (Weeks 26-28)**

Complete penetration testing and vulnerability assessment. Finalize audit logging with 6-year retention and tamper-proof storage. Complete HIPAA risk assessment documentation. Conduct load testing (target: 10,000 concurrent verifications). Implement rate limiting and abuse prevention. Complete SOC 2 Type I readiness assessment. Finalize BAAs with all vendors. Deploy to production with blue-green deployment strategy. Onboard first 5 pilot clients.

**Deliverable:** Production-ready, HIPAA-compliant system with pilot clients.

---

## H) AI platform instructions: configuring agents for eligibility tasks

**Claude/GPT-4 agent configuration for eligibility verification**

**Agent 1: EDI 271 Response Parser**

SYSTEM PROMPT:

You are an expert healthcare EDI specialist. Your task is to parse raw X12 EDI 271 (Eligibility/Benefit Response) transactions and extract structured data.

RULES:

- Parse all EB (Eligibility/Benefit) segments to extract coverage details
- Map service type codes to human-readable service names (30=Health Benefit Plan Coverage, 33=Chiropractic, 47=Hospital, 48=Hospital Inpatient, 50=Hospital Outpatient, 86=Emergency Services, 88=Pharmacy, 98=Professional Physician)
- Extract monetary amounts from EB segments (copay=B, coinsurance=A, deductible=C)
- Parse DTP segments for coverage effective/termination dates
- Identify AAA error segments and translate error codes
- Handle HL hierarchy: 2000A=Payer, 2000B=Provider, 2000C=Subscriber, 2000D=Dependent
- Output JSON matching the schema provided below
- Set null for any field that cannot be determined; include reason in "notes" array
- Flag data inconsistencies (e.g., termination date before effective date)

OUTPUT SCHEMA:

```
{
 "coverage_status": "ACTIVE|INACTIVE|TERMINATED|UNKNOWN",
 "plan_name": string,
 "plan_type": "HMO|PPO|EPO|POS|HDHP|OTHER",
 "effective_date": "YYYY-MM-DD",
 "termination_date": "YYYY-MM-DD|null",
 "subscriber": { "name": string, "member_id": string, "group_number": string },
 "benefits": {
  "deductible": {
   "individual_in_network": { "total": number, "remaining": number },
   "family_in_network": { "total": number, "remaining": number }
  },
  "oop_maximum": {
   "individual_in_network": { "total": number, "remaining": number }
  },
  "copays": {
   "pcp_visit": number, "specialist_visit": number,
   "er_visit": number, "urgent_care": number
  },
  "coinsurance": { "in_network": number, "out_of_network": number }
 },
 "prior_auth_required": boolean,
 "pcp_required": boolean,
 "coordination_of_benefits": string|null,
 "errors": [{ "code": string, "description": string }],
```

```
  "notes": [string],
  "confidence_score": 0.0-1.0
}
```

### Agent 2: Patient Cost Estimator

SYSTEM PROMPT:
You are a healthcare financial counselor AI. Given structured eligibility data and
a scheduled procedure (CPT code), calculate the estimated patient responsibility.

PROCESS (chain-of-thought):
1. Confirm coverage is active for the service date
2. Determine if the service requires prior authorization
3. Check if the provider is in-network or out-of-network
4. Apply deductible: if not met, patient pays up to remaining deductible amount
5. After deductible, apply copay OR coinsurance (not both, per plan rules)
6. Check against out-of-pocket maximum — patient never pays above OOP max
7. For services with both facility and professional fees, calculate separately
8. Present low/mid/high estimates to account for uncertainty

IMPORTANT:
- State clearly when estimates are uncertain
- Flag if the CPT code is typically excluded or limited under the plan type
- Note if coordination of benefits may reduce patient responsibility
- Include the mathematical breakdown showing your work

### Agent 3: Coverage Anomaly Detector

SYSTEM PROMPT:
You are a healthcare coverage auditor. Analyze eligibility verification results
to identify potential issues that could lead to claim denials.

FLAG THE FOLLOWING:
- Policy termination within 30 days of service date
- Coordination of benefits indicators without secondary payer on file
- Prior authorization required but not initiated
- Out-of-network provider for HMO/EPO plans (likely denied)
- Deductible nearly exhausted (patient may have large OOP responsibility)
- Plan type mismatches (e.g., dental CPT under medical-only plan)
- Missing or inconsistent member ID formats per payer standards
- Coverage gaps between termination of one policy and effective date of another

OUTPUT: JSON array of {severity: HIGH|MEDIUM|LOW, issue: string,
recommendation: string, denial_risk_percentage: number}

**RAG pipeline configuration**

Ingest the following document types into the vector database: payer provider manuals (one per major payer, updated quarterly), plan benefit summaries (SPDs for top 50 plans by volume), clinical coverage policies (medical necessity criteria by payer), fee schedules (allowed amounts by CPT code and payer), prior authorization requirement lists (services requiring PA by payer), and denial reason code mappings (CARC/RARC codes to plain language). Chunk documents at **512 tokens with 50-token overlap**. Use BGE-M3 embeddings. Retrieve top 5 chunks per query with **minimum similarity threshold of 0.75**.

---

## I) Dashboard and reporting specifications

### KPI framework

The dashboard tracks three tiers of metrics across two views — operational (real-time for staff) and executive (aggregated for leadership).

**Tier 1 — Operational metrics (real-time):**

- **Verification completion rate** — % of scheduled patients verified before appointment (target: >98%)
- **Average verification turnaround** — seconds from trigger to complete result (target: <5 seconds real-time, <24 hours batch)
- **Bot success rate** — % verifications completed without human intervention (target: >85%, matching Jindal's 70%+ claim)

- **Payer response time** — average 271 response latency by payer (benchmark: <5 seconds per CAQH CORE)
- **Exception queue depth** — number of verifications awaiting human review
- **Active coverage rate** — % of verifications returning active coverage

**Tier 2 — Financial impact metrics (daily/weekly):**

- **Eligibility-related denial rate** — % of denials attributed to eligibility issues (target: <5%)
- **Clean claim rate** — % of claims paid on first submission (target: >95%)
- **Cost per verification** — automated ($2.50 average) vs manual ($6.50+ average)
- **Monthly cost savings** — (manual_cost - automated_cost) × verification_volume
- **Coverage discovery revenue** — additional revenue from found coverage for self-pay patients
- **Point-of-service collection rate** — % of patient responsibility collected at time of service

**Tier 3 — Strategic metrics (monthly/quarterly):**

- **Days in accounts receivable** (target: <35 days)
- **Net collection rate** (target: >95%)
- **Denial trend analysis** — patterns by payer, denial reason, provider, service type
- **Payer performance scorecard** — response times, error rates, coverage accuracy by payer
- **ROI dashboard** — total cost savings, staff hours recaptured, denial reduction value

**Dashboard layout specification**

The **Eligibility Verification Command Center** uses a responsive React layout with four sections. The **top bar** contains date range selector, organization/location filter, and payer filter. The **KPI ribbon** displays four headline cards: verification rate (%), average response time (seconds), eligibility denial rate (%), and automation success rate (%). The **middle panel** splits into a 30-day verification volume trend line chart (left) and a payer response time heatmap by payer × time of day (right). The **bottom panel** shows a denial reasons breakdown bar chart (left) and a live exception queue table with patient name, payer, error type, time waiting, and action buttons (right).

---

## J) EHR integration specifications

### Integration matrix by EHR

| EHR | Auth Method | Eligibility Trigger | Primary API |
|---|---|---|---|
| **Epic** | SMART on FHIR OAuth 2.0 | Appointment/ADT webhook + SMART app launch | `/api/FHIR/R4/Coverage` `/api/FHIR/R4/Patient` |
| **Athenahealth** | OAuth 2.0 (2-legged backend) | `POST /v1/{practiceid}/insurances/{insuranceid}/eligibility` | Proprietary REST + Certified FHIR |
| **eClinicalWorks** | OAuth 2.0 | Clearinghouse integration (Waystar/Trizetto) + FHIR | `/fhir/r4/{practice_code}` (250 calls/min) |
| **Cerner/Oracle** | SMART on FHIR v1/v2 | Subscription API + ADT listener | `/r4/{tenant-id}/Coverag` `/r4/{tenant-id}/Patient` |
| **NextGen** | OAuth 2.0 + PKCE | 800+ REST API routes + Mirth Connect | JSON RESTful + FHIR R |
| **DrChrono** | OAuth 2.0 (authorization_code) | Webhooks (appointment created/modified) | `/api/patients`, `/api/insurances` (500 calls/hr) |
| **Kareo/Tebra** | SOAP API | Built-in eligibility during scheduling | SOAP endpoints |
| **AdvancedMD** | OAuth 2.0 | REST API triggers | FHIR R4 Bulk API via Apigee HealthAPIx |
| **Practice Fusion** | SMART on FHIR OAuth 2.0 + PKCE | FHIR R4 resources | Standard FHIR R4 |

### HL7 v2 ADT integration via Mirth Connect

For EHRs that primarily communicate via HL7 v2 messaging, deploy **Mirth Connect** (NextGen-owned, powers 1/3 of US public HIEs) as the integration engine. Configure TCP/IP MLLP listeners for ADT messages. The key trigger is **ADT^A04** (Register Patient), which contains the **IN1 segment** with plan ID, group number, insurer name, and member ID. On receiving ADT^A04, the Mirth channel extracts PID (patient demographics) and IN1 (insurance info) segments, constructs an X12 270, submits to the clearinghouse, receives the 271, and routes the parsed response back to the dashboard or EHR.

**SMART on FHIR embedded app**

The eligibility verification widget embeds directly into clinician workflows via SMART on FHIR. When a clinician opens a patient chart, the app auto-launches with patient context via the EHR Launch flow. The app reads the Coverage resource, submits a CoverageEligibilityRequest, and displays the response inline — all without the clinician leaving their EHR. Use `fhir-client.js` for the JavaScript SDK. Register the app through each EHR's vendor program: Epic Showroom (vendorservices.epic.com), Oracle Healthcare Marketplace, Athenahealth Marketplace, and NextGen Developer Program.

**Certification requirements**

Epic certification is the most rigorous — requiring security, performance, and interoperability testing through the Showroom program. Athenahealth requires a Platform Services contract and BAA. Oracle Health requires registration in code Console and completion of the Oracle Validated Integration process (2+ weeks). All SMART on FHIR apps must meet the ONC Cures Act (21st Century Cures) requirements for information blocking prevention.

---

## Conclusion: from brief to build

This technical brief provides a complete blueprint for building StaffVerify as a market-competitive eligibility verification platform. Three decisions will most impact success. **First, the clearinghouse strategy** — starting with Availity's free tier for broadest payer reach plus Stedi for modern EDI parsing gives the best cost-to-coverage ratio. **Second, the AI-first architecture** — using Claude/GPT-4 with RAG for benefits interpretation rather than hard-coded rules creates a system that improves with every verification and adapts to payer-specific nuances that break rigid parsers. **Third, the phased approach to payer connectivity** — EDI 270/271 covers 94%+ of verifications today, FHIR adoption will accelerate through 2027 as CMS-0057-F mandates take effect, and RPA fills the gaps for payers without electronic channels.

The most underappreciated competitive advantage Staffingly holds is its **800+ provider client base** generating massive verification volume. Every verification trains the AI models, every denial feeds the prediction engine, and every payer response enriches the rules database. Scale creates a data flywheel that standalone tools cannot replicate. The build plan above targets a **28-week timeline** from infrastructure setup to pilot launch, with the EDI 270/271 pipeline (Sprint 2) and frontend dashboard (Sprint 4) delivering usable value by week 12 — fast enough to demonstrate ROI to pilot clients while the AI and RPA layers mature in subsequent sprints.